UNIVERSITE DE KINSHASA

FACULTE DES SCIENCES ET TECHNOLOGIES

Département de Mathématiques, Statistique et informatique

B.P. 190 KINSHASA XI

EXAMEN DE MACHINE LEARNING

SUJET : **OPTIMISATION D'HYPERPARAMETRES DISTRIBUEE : IMPLEMENTATION VIA OPTUNA/RAY TUNE**

Travail présenté par :

AZIDAMA EGAO BELVINE

MULANGA MUYA ASNATH

KITAMBALA NKETO ELODIE

Année – Académique : 2025 – 2026

Année – Académique : 2025 – 2026

TABLE DES MATIERES

1. INTRODUCTION 3
2. CONCEPTS THÉORIQUES 5
2.1. Optimisation des hyperparamètres 5
2.2. Optuna 5
2.2.1. Samplers 5
2.2.2. Pruners 5
2.3. Ray Tune 6
2.3.1. Search Algorithms 6
2.3.2. Schedulers 6
2.4. Optimisation distribuée 6
3. ÉTAT DE L'ART ET ANALYSE CRITIQUE 7
3.1. Grid Search 7
3.2. Random Search 7
3.3. Optimisation bayésienne 7
3.4. Pruning 7
3.5. Optimisation distribuée 8
3.6. Optuna et Ray Tune 8
3.7. Analyse critique 8
4. TABLEAU COMPARATIF D'OPTUNA ET RAY TUNE 10
4.1. Avantages et limites 10
Optuna 10
Ray Tune 11
4.2. Choix pour le TP 11
5. IMPLÉMENTATION AVEC OPTUNA 12
5.1. Objectif 12
5.2. Définition de l'espace de recherche 12
5.3. Déroulement de l'expérimentation 12
5.4. Analyse des résultats ..... **Erreur ! Signet non défini.**
6. IMPLÉMENTATION AVEC RAY TUNE 14
6.1. Objectif 14
6.2. Configuration de Ray Tune 14
6.3. Utilisation d'ASHAScheduler 14
6.4. Utilisation de Population Based Training (PBT) 14
6.5. Exécution parallèle et gestion des ressources 14

6.6. Analyse des résultats 15
7. INTÉGRATION ET ANALYSE DES RÉSULTATS 16
7.1. Intégration d'Optuna dans Ray Tune 16
7.2. Visualisation de la convergence 16
7.3. Importance des hyperparamètres 16
7.4. Comparaison des résultats 17
7.5. Discussion 17
8. CONCLUSION 18

1. INTRODUCTION

L'apprentissage automatique (*Machine Learning*) occupe aujourd'hui une place importante dans le développement de solutions informatiques capables d'analyser des données, d'identifier des tendances et d'effectuer des prédictions. La performance d'un modèle de Machine Learning dépend non seulement de l'algorithme choisi et des données utilisées, mais également des paramètres qui contrôlent son apprentissage. Ces paramètres, appelés **hyperparamètres**, doivent être définis avant ou autour de la phase d'entraînement.

Le choix des valeurs des hyperparamètres représente donc une étape importante dans la conception d'un modèle performant. Par exemple, dans un réseau de neurones, le taux d'apprentissage, le nombre de couches, le nombre de neurones ou la taille des lots peuvent influencer les résultats obtenus. Une mauvaise configuration peut entraîner un apprentissage insuffisant, un surapprentissage ou une augmentation inutile du temps de calcul.

La recherche manuelle des meilleures configurations devient cependant difficile lorsque le nombre d'hyperparamètres et de valeurs possibles augmente. Les méthodes classiques telles que la recherche par grille (*Grid Search*) peuvent nécessiter un grand nombre d'expérimentations. De même, la recherche aléatoire (*Random Search*) peut nécessiter plusieurs essais avant d'obtenir une configuration satisfaisante. Ces difficultés ont favorisé le développement de méthodes automatisées permettant d'explorer plus efficacement les espaces d'hyperparamètres.

Par ailleurs, l'augmentation de la complexité des modèles et du nombre d'expérimentations entraîne des besoins importants en ressources de calcul. L'**optimisation distribuée** constitue alors une solution permettant de répartir plusieurs essais sur différentes ressources afin de réaliser des expérimentations en parallèle. Cette approche peut contribuer à réduire le temps nécessaire à la recherche des meilleures configurations.

Dans ce contexte, **Optuna** et **Ray Tune** sont deux outils permettant d'automatiser et d'améliorer l'optimisation des hyperparamètres. Optuna propose notamment des **samplers**, chargés de rechercher les configurations à tester, ainsi que des **pruners**, permettant d'arrêter les essais qui se révèlent peu prometteurs. Ray Tune, quant à lui, fournit des **search algorithms** pour guider la recherche et des **schedulers** pour gérer les essais pendant leur exécution. Il permet également de faciliter la parallélisation et la distribution des expérimentations.

L'objectif de ce TP est donc d'étudier et de mettre en œuvre une **optimisation d'hyperparamètres distribuée à travers Optuna et Ray Tune**. Il s'agit d'abord de comprendre les principaux concepts liés à ces deux outils, puis de réaliser des expérimentations permettant d'observer leur fonctionnement dans un contexte réel de Machine Learning.

Le travail consiste notamment à mettre en place une optimisation avec **Optuna**, puis à utiliser **Ray Tune** pour distribuer les essais et expérimenter des mécanismes tels que **ASHAScheduler** et **Population Based Training**. Une intégration d'**OptunaSearch dans Ray Tune** sera également étudiée afin d'observer comment les mécanismes de recherche d'Optuna peuvent être combinés avec les capacités d'exécution et de gestion des ressources de Ray Tune.

Enfin, les résultats obtenus permettront de comparer les différentes approches selon plusieurs critères, notamment les performances du modèle, le temps d'exécution, le nombre d'essais réalisés et l'utilisation des ressources. Cette expérimentation permettra ainsi de mieux comprendre l'intérêt de l'optimisation automatique et distribuée des hyperparamètres dans les projets de Machine Learning.

2. CONCEPTS THÉORIQUES

2.1. Optimisation des hyperparamètres

Les hyperparamètres sont des paramètres définis avant l'entraînement d'un modèle de Machine Learning. Ils contrôlent certains aspects du processus d'apprentissage et peuvent avoir une influence importante sur les performances du modèle.

L'optimisation des hyperparamètres consiste à rechercher automatiquement les valeurs ou les configurations qui permettent d'obtenir les meilleures performances selon une métrique donnée. Cette recherche peut concerner, par exemple, le taux d'apprentissage, le nombre de couches d'un réseau neuronal, le nombre de neurones, la taille du lot (*batch size*) ou le nombre d'arbres d'un modèle de type Random Forest.

Lorsque le nombre de paramètres et de configurations possibles augmente, une recherche manuelle devient difficile et coûteuse. L'utilisation de frameworks spécialisés permet alors d'automatiser cette recherche et de réduire le temps consacré aux expérimentations.

2.2. Optuna

Optuna est un framework d'optimisation des hyperparamètres destiné aux modèles de Machine Learning. Il fonctionne en réalisant plusieurs essais (*trials*) au cours desquels différentes configurations d'hyperparamètres sont évaluées.

2.2.1. Samplers

Le **sampler** est le mécanisme chargé de proposer les valeurs des hyperparamètres pour les différents essais.

Parmi les samplers disponibles dans Optuna, on peut citer :

- **RandomSampler**, qui sélectionne les configurations de manière aléatoire ;
- **TPESampler**, qui utilise les résultats des essais précédents afin d'orienter la recherche vers des configurations potentiellement plus performantes.

Le sampler joue donc un rôle essentiel dans la stratégie de recherche des hyperparamètres.

2.2.2. Pruners

Le **pruner** permet d'interrompre prématurément un essai lorsque ses performances intermédiaires indiquent qu'il est peu susceptible d'obtenir un bon résultat.

Cette fonctionnalité permet de réduire le temps de calcul et d'éviter de consacrer des ressources à des configurations peu prometteuses.

2.3. Ray Tune

Ray Tune est une bibliothèque de l'écosystème Ray destinée à l'optimisation des hyperparamètres et à la gestion d'expérimentations de Machine Learning. Elle permet notamment d'exécuter plusieurs essais en parallèle et de gérer les ressources disponibles.

2.3.1. Search Algorithms

Les **search algorithms** déterminent les configurations d'hyperparamètres qui doivent être testées. Ils permettent de définir la stratégie de recherche utilisée pendant l'optimisation.

Ray Tune peut notamment être associé à différents algorithmes de recherche, dont **OptunaSearch**, qui permet d'utiliser la stratégie de recherche d'Optuna dans l'environnement Ray Tune.

2.3.2. Schedulers

Les **schedulers** permettent de gérer les essais pendant leur exécution. Ils peuvent notamment décider quels essais doivent continuer, être arrêtés ou recevoir davantage de ressources selon leur évolution.

Dans ce TP, deux mécanismes sont particulièrement étudiés :

- **ASHAScheduler**, qui permet d'arrêter rapidement certains essais peu performants afin de consacrer les ressources aux essais plus prometteurs ;
- **Population Based Training (PBT)**, qui permet de faire évoluer les configurations des essais pendant leur entraînement en favorisant les configurations performantes.

2.4. Optimisation distribuée

L'optimisation distribuée consiste à répartir les différents essais d'optimisation sur plusieurs ressources de calcul. Au lieu d'exécuter les essais les uns après les autres, plusieurs expérimentations peuvent être réalisées simultanément.

Cette approche permet principalement de réduire le temps global d'expérimentation et de mieux exploiter les ressources disponibles, notamment les processeurs et les GPU.

Dans le cadre de ce TP, Ray Tune est utilisé pour étudier cette approche distribuée et pour analyser la manière dont plusieurs essais peuvent être exécutés et gérés simultanément.

3. ÉTAT DE L'ART ET ANALYSE CRITIQUE

L'optimisation des hyperparamètres est devenue une étape importante dans le développement des modèles de Machine Learning. Plusieurs méthodes ont été proposées afin d'automatiser la recherche des configurations les plus performantes.

3.1. Grid Search

La méthode **Grid Search** consiste à définir à l'avance plusieurs valeurs possibles pour chaque hyperparamètre, puis à tester toutes les combinaisons possibles.

Cette méthode est simple à comprendre et permet d'explorer systématiquement l'espace de recherche. Cependant, le nombre d'expérimentations augmente rapidement lorsque plusieurs hyperparamètres sont utilisés. Elle peut donc devenir coûteuse en temps de calcul.

3.2. Random Search

La méthode **Random Search** sélectionne aléatoirement différentes configurations d'hyperparamètres parmi celles définies dans l'espace de recherche.

Contrairement à Grid Search, elle ne teste pas nécessairement toutes les combinaisons. Elle peut ainsi trouver rapidement de bonnes configurations lorsque certains hyperparamètres sont plus importants que d'autres. Cependant, les résultats dépendent en partie des configurations sélectionnées aléatoirement et certaines expérimentations peuvent être peu utiles.

3.3. Optimisation bayésienne

L'optimisation bayésienne utilise les résultats des expérimentations précédentes pour orienter les essais suivants vers des configurations susceptibles d'être meilleures.

Cette approche permet généralement d'éviter une exploration totalement aléatoire de l'espace de recherche. Elle est particulièrement intéressante lorsque l'évaluation d'un modèle demande beaucoup de temps ou de ressources.

Le **TPESampler** d'Optuna constitue notamment une approche permettant d'orienter progressivement la recherche à partir des résultats obtenus lors des essais précédents.

3.4. Pruning

Le *pruning* consiste à interrompre un essai avant sa fin lorsque ses performances intermédiaires sont insuffisantes.

Cette technique permet d'éviter de poursuivre l'entraînement de modèles qui ont peu de chances de devenir performants. Elle contribue ainsi à réduire le temps de calcul et la consommation des ressources.

Optuna propose des **pruners**, tandis que Ray Tune dispose de mécanismes de planification tels que **ASHAScheduler** permettant également d'arrêter les essais peu prometteurs.

3.5. Optimisation distribuée

Avec l'augmentation du nombre d'essais et de la complexité des modèles, l'exécution séquentielle des expérimentations peut devenir très longue. L'optimisation distribuée répond à cette difficulté en permettant d'exécuter plusieurs essais simultanément sur différentes ressources de calcul.

Ray Tune est particulièrement adapté à cette approche puisqu'il permet de gérer l'exécution parallèle des essais et l'allocation des ressources disponibles.

3.6. Optuna et Ray Tune

Optuna est principalement orienté vers la recherche efficace des meilleurs hyperparamètres. Il fournit notamment des samplers pour proposer les configurations et des pruners pour interrompre les essais peu prometteurs.

Ray Tune se concentre davantage sur la gestion et l'exécution des expérimentations à grande échelle. Il permet de paralléliser les essais, de gérer les ressources et d'utiliser différents algorithmes de recherche et schedulers.

Les deux outils peuvent également être combinés. Par exemple, **OptunaSearch** permet d'utiliser la stratégie de recherche d'Optuna dans l'environnement Ray Tune.

3.7. Analyse critique

Les méthodes classiques comme Grid Search présentent l'avantage d'être simples et systématiques, mais elles deviennent rapidement coûteuses lorsque l'espace de recherche est important. Random Search améliore l'efficacité de l'exploration, mais ne garantit pas que les essais les plus pertinents seront sélectionnés rapidement.

L'optimisation bayésienne permet une recherche plus orientée grâce à l'utilisation des résultats précédents. Toutefois, lorsque le nombre d'expérimentations devient important, la capacité à exécuter plusieurs essais en parallèle devient également essentielle.

Dans ce contexte, Optuna offre des mécanismes efficaces pour rechercher et sélectionner les hyperparamètres, tandis que Ray Tune apporte des fonctionnalités supplémentaires pour la parallélisation, la distribution des essais et la gestion des ressources. L'utilisation

combinée des deux outils peut donc permettre de bénéficier à la fois d'une stratégie de recherche efficace et d'une exécution distribuée.

Ainsi, le choix entre les différentes approches dépend principalement de la taille de l'espace de recherche, du coût d'entraînement du modèle et des ressources de calcul disponibles. C'est dans cette perspective que le présent TP expérimente Optuna, Ray Tune et leur intégration afin d'évaluer leurs avantages dans un contexte pratique.

4. TABLEAU COMPARATIF D'OPTUNA ET RAY TUNE

Critère	Optuna	Ray Tune
Objectif principal	Optimisation automatique des hyperparamètres	Optimisation et gestion d'expérimentations à grande échelle
Type d'outil	Framework d'optimisation des hyperparamètres	Bibliothèque d'expérimentation et d'optimisation distribuée
Recherche des configurations	Utilisation de **samplers**	Utilisation de **search algorithms**
Gestion des essais	Utilisation de **pruners** pour interrompre les essais peu prometteurs	Utilisation de **schedulers** pour gérer les essais pendant leur exécution
Optimisation distribuée	Possible avec une configuration adaptée	Forte intégration avec l'exécution distribuée de Ray
Parallélisation	Permet l'exécution de plusieurs essais	Permet de gérer efficacement plusieurs essais en parallèle
Facilité d'utilisation	Relativement simple et spécialisé	Plus riche, mais peut être plus complexe
Flexibilité	Très adapté à l'optimisation des hyperparamètres	Adapté à l'optimisation et à la gestion d'expérimentations complexes
Gestion des ressources	Principalement grâce au pruning et à la gestion des essais	Gestion avancée des ressources et des essais distribués
Cas d'utilisation	Recherche efficace des meilleurs hyperparamètres	Expérimentations à grande échelle et optimisation distribuée

4.1. Avantages et limites

Optuna

Avantages :

- interface relativement simple ;
- plusieurs stratégies de recherche disponibles ;
- possibilité d'arrêter les essais peu prometteurs grâce aux pruners ;
- adapté à l'optimisation de modèles de Machine Learning.

Limites :

- la gestion des ressources distribuées est moins centrale ;
- la distribution à grande échelle nécessite généralement une infrastructure complémentaire.

Ray Tune

Avantages :

- exécution parallèle de nombreux essais ;
- gestion des ressources de calcul ;
- prise en charge de plusieurs algorithmes de recherche et schedulers ;
- adapté aux expérimentations distribuées.

Limites :

- configuration plus complexe pour un débutant ;
- utilisation de ressources plus importante selon l'environnement ;
- certaines fonctionnalités peuvent être inutiles pour de petites expérimentations.

4.2. Choix pour le TP

Dans le cadre de ce TP, les deux outils seront étudiés afin de mettre en évidence leurs rôles complémentaires.

Optuna sera utilisé pour expérimenter l'optimisation des hyperparamètres à travers ses samplers et ses pruners. Ray Tune sera ensuite utilisé pour étudier la parallélisation et la distribution des essais, notamment avec **ASHAScheduler** et **Population Based Training (PBT)**.

Enfin, l'intégration d'**OptunaSearch dans Ray Tune** permettra d'étudier la combinaison d'une stratégie de recherche issue d'Optuna avec les capacités de gestion et d'exécution distribuée de Ray Tune.

Les résultats seront comparés notamment selon les performances obtenues, le temps d'exécution, le nombre d'essais réalisés et l'utilisation des ressources.

5. IMPLÉMENTATION AVEC OPTUNA

5.1. Objectif

Cette partie consiste à mettre en œuvre Optuna afin d'automatiser la recherche des meilleurs hyperparamètres d'un modèle de Machine Learning. L'objectif est de tester différentes configurations, d'évaluer leurs performances et d'identifier celle qui fournit le meilleur résultat selon une métrique définie.

5.2. Définition de l'espace de recherche

L'espace de recherche correspond à l'ensemble des valeurs que peuvent prendre les hyperparamètres pendant l'optimisation. Pour cette expérimentation, les hyperparamètres peuvent notamment inclure :

- **Taux d'apprentissage (*learning rate*) :** contrôle l'importance des mises à jour des paramètres du modèle.
- **Nombre de couches :** détermine la profondeur du réseau neuronal.
- **Nombre de neurones :** détermine la taille des couches cachées.
- **Taille du lot (*batch size*) :** indique le nombre d'exemples traités lors d'une étape d'apprentissage.
- **Nombre d'époques (*epochs*) :** correspond au nombre de passages sur les données d'entraînement.

Les valeurs possibles sont définies dans le programme et explorées automatiquement par Optuna.

5.3. Déroulement de l'expérimentation

L'implémentation suit les étapes suivantes :

1. Préparer les données d'entraînement et de test.
2. Définir le modèle de Machine Learning à optimiser.
3. Définir la fonction objectif qui entraîne le modèle et mesure ses performances.
4. Déclarer l'espace de recherche des hyperparamètres.
5. Créer une étude Optuna et lancer plusieurs essais.
6. Identifier la configuration ayant obtenu le meilleur résultat.
7. Enregistrer et analyser les résultats obtenus.

5.4. Exécution des essais

Comme, Nous avons configuré optuina avec n_trials = 30

chaque trial correspond à un entraînement avec une combinaison différente des hyperparamètres

Comme résultats (accuracy + paramètres) sont affichés à chaque étape.

```
[I 2026-10-08 22:44:53,543] A new study created in memory with name: no-name-84cdde5c-6adf-47aa-96a5-12109b1dd50d
[I 2026-10-08 22:45:25,843] Trial 0 finished with value: 0.9039166666666667 and parameters: {'n_units': 90, 'dropout': 0.23699026996261982, 'lr': 0.001
1480338748640904}. Best is trial 0 with value: 0.9039166666666667.
Trial 0 terminé avec valeur=0.9039166666666667 et params={'n_units': 90, 'dropout': 0.23699026996261982, 'lr': 0.0011480338748640904}
[I 2026-10-08 22:45:58,367] Trial 1 finished with value: 0.8910833333333333 and parameters: {'n_units': 105, 'dropout': 0.25653502858545224, 'lr': 0.00
0698171144403426 3}. Best is trial 0 with value: 0.9039166666666667.
Trial 1 terminé avec valeur=0.8910833333333333 et params={'n_units': 105, 'dropout': 0.25653502858545224, 'lr': 0.0006981711444034263}
[I 2026-10-08 22:46:30,196] Trial 2 finished with value: 0.7663333333333333 and parameters: {'n_units': 41, 'dropout': 0.4598810578834312, 'lr': 0.0008
024390084777414}. Best is trial 0 with value: 0.9039166666666667.
Trial 2 terminé avec valeur=0.7663333333333333 et params={'n_units': 41, 'dropout': 0.4598810578834312, 'lr': 0.0008024390084777414}
[I 2026-10-08 22:47:01,473] Trial 3 finished with value: 0.8790833333333333 and parameters: {'n_units': 101, 'dropout': 0.4375433720647164, 'lr': 0.001
1137030294880311}. Best is trial 0 with value: 0.9039166666666667.
Trial 3 terminé avec valeur=0.8790833333333333 et params={'n_units': 101, 'dropout': 0.4375433720647164, 'lr': 0.0011137030294880311}
[I 2026-10-08 22:47:33,848] Trial 4 finished with value: 0.8838333333333334 and parameters: {'n_units': 113, 'dropout': 0.33889910398209533, 'lr': 0.00
030936340719518496}. Best is trial 0 with value: 0.9039166666666667.
Trial 4 terminé avec valeur=0.8838333333333334 et params={'n_units': 113, 'dropout': 0.33889910398209533, 'lr': 0.0003093634071951849 6}
[I 2026-10-08 22:48:06,266] Trial 5 finished with value: 0.8631666666666666 and parameters: {'n_units': 124, 'dropout': 0.379311967566632, 'lr': 0.0001
441327357938191}. Best is trial 0 with value: 0.9039166666666667.
Trial 5 terminé avec valeur=0.8631666666666666 et params={'n_units': 124, 'dropout': 0.379311967566632, 'lr': 0.0001441327357938191}
[I 2026-10-08 22:48:38,890] Trial 6 finished with value: 0.8841666666666667 and parameters: {'n_units': 121, 'dropout': 0.25826698959074945, 'lr': 0.00
0219005330513 2221}. Best is trial 0 with value: 0.9039166666666667.
Trial 6 terminé avec valeur=0.8841666666666667 et params={'n_units': 121, 'dropout': 0.25826698959074945, 'lr': 0.0002190053305132221}
[I 2026-10-08 22:49:11,614] Trial 7 finished with value: 0.7960833333333334 and parameters: {'n_units': 91, 'dropout': 0.499953390947953, 'lr': 0.00256
7938869173492}. Best is trial 0 with value: 0.9039166666666667.
Trial 7 terminé avec valeur=0.7960833333333334 et params={'n_units': 91, 'dropout': 0.499953390947953, 'lr': 0.002567938869173492}
[I 2026-10-08 22:49:44,433] Trial 8 finished with value: 0.8439166666666666 and parameters: {'n_units': 86, 'dropout': 0.3251601807634584, 'lr': 0.0047
260613615779995}. Best is trial 0 with value: 0.9039166666666667.
Trial 8 terminé avec valeur=0.8439166666666666 et params={'n_units': 86, 'dropout': 0.3251601807634584, 'lr': 0.004726061361577999 5}
[I 2026-10-08 22:50:16,998] Trial 9 finished with value: 0.873 and parameters: {'n_units': 86, 'dropout': 0.22242381146228096, 'lr': 0.0001922690431364
0345}. Best is trial 0 with value: 0.9039166666666667.
Trial 9 terminé avec valeur=0.873 et params={'n_units': 86, 'dropout': 0.22242381146228096, 'lr': 0.00019226904313640345}
[I 2026-10-08 22:50:48,881] Trial 10 finished with value: 0.8424166666666667 and parameters: {'n_units': 33, 'dropout': 0.24767450594302826, 'lr': 0.00
05377862336330857}. Best is trial 0 with value: 0.9039166666666667.
Trial 10 terminé avec valeur=0.8424166666666667 et params={'n_units': 33, 'dropout': 0.24767450594302826, 'lr': 0.0005377862336330857}
```

Nous constatons que

```
[I 2026-10-08 22:57:19,522] Trial 22 finished with value: 0.8495833333333334 and parameters: {'n_units': 120, 'dropout': 0.3141731444837558, 'lr': 0.00
4799002717475048}. Best is trial 14 with value: 0.9108333333333334.
Trial 22 terminé avec valeur=0.8495833333333334 et params={'n_units': 120, 'dropout': 0.3141731444837558, 'lr': 0.004799002717475048}
[I 2026-10-08 22:57:52,188] Trial 23 finished with value: 0.9130833333333334 and parameters: {'n_units': 124, 'dropout': 0.27676696694632474, 'lr': 0.0
010369783150077035}. Best is trial 23 with value: 0.9130833333333334.
Trial 23 terminé avec valeur=0.9130833333333334 et params={'n_units': 124, 'dropout': 0.27676696694632474, 'lr': 0.0010369783150077035}
[I 2026-10-08 22:58:24,893] Trial 24 finished with value: 0.8740833333333333 and parameters: {'n_units': 127, 'dropout': 0.24751909480869516, 'lr': 0.0
072853385097165595}. Best is trial 23 with value: 0.9130833333333334.
Trial 24 terminé avec valeur=0.8740833333333333 et params={'n_units': 127, 'dropout': 0.24751909480869516, 'lr': 0.0072853385097165595}
[I 2026-10-08 22:58:57,225] Trial 25 finished with value: 0.9050833333333334 and parameters: {'n_units': 117, 'dropout': 0.27627447952629625, 'lr': 0.0
012843317561697354}. Best is trial 23 with value: 0.9130833333333334.
Trial 25 terminé avec valeur=0.9050833333333334 et params={'n_units': 117, 'dropout': 0.27627447952629625, 'lr': 0.0012843317561697354}
[I 2026-10-08 22:59:29,676] Trial 26 finished with value: 0.908 and parameters: {'n_units': 110, 'dropout': 0.24214188481908774, 'lr': 0.00234825165566
2932}. Best is trial 23 with value: 0.9130833333333334.
Trial 26 terminé avec valeur=0.908 et params={'n_units': 110, 'dropout': 0.24214188481908774, 'lr': 0.002348251655662932}
[I 2026-10-08 23:00:02,215] Trial 27 finished with value: 0.8968333333333334 and parameters: {'n_units': 124, 'dropout': 0.32395339057470335, 'lr': 0.0
023079933574024 52}. Best is trial 23 with value: 0.9130833333333334.
Trial 27 terminé avec valeur=0.8968333333333334 et params={'n_units': 124, 'dropout': 0.32395339057470335, 'lr': 0.002307993357402452}
[I 2026-10-08 23:00:34,658] Trial 28 finished with value: 0.81275 and parameters: {'n_units': 109, 'dropout': 0.2701444001031501, 'lr': 0.0095358234702
51665}. Best is trial 23 with value: 0.9130833333333334.
Trial 28 terminé avec valeur=0.81275 et params={'n_units': 109, 'dropout': 0.2701444001031501, 'lr': 0.009535823470251665}
[I 2026-10-08 23:01:07,600] Trial 29 finished with value: 0.8998333333333334 and parameters: {'n_units': 127, 'dropout': 0.23405992069678436, 'lr': 0.0
018322915801394191}. Best is trial 23 with value: 0.9130833333333334.
Trial 29 terminé avec valeur=0.8998333333333334 et params={'n_units': 127, 'dropout': 0.23405992069678436, 'lr': 0.0018322915801394191}
Meilleurs hyperparamètres: {'n_units': 124, 'dropout': 0.27676696694632474, 'lr': 0.0010369783150077035}
Meilleure accuracy: 0.9130833333333334
```

Meilleurs hyperparamètres :

n_units = <u>124</u> c'est-à-dire le nombre de neurone est suffisant pour capturer la complexité des données MNIST sans surapprentissage.
lr = <u>0.00139</u> cest à dire le learning rate optimal est relativement petit

ce qui favorise une convergence stable
dropout = <u>0.2767</u> cest à dire le taux de dropout choisi montre un bon équilibre entre régularisation et la performance
Meilleure accuracy trouvée est de : 91 %

Cela montre que l'optimisation nous a permis d'explorer plusieurs combinaisons et de retenir la configuration la plus performante

6. IMPLÉMENTATION AVEC RAY TUNE

6.1. Objectif

Cette partie consiste à utiliser Ray Tune pour automatiser l'optimisation des hyperparamètres et étudier l'exécution parallèle des essais. L'objectif est de comparer différentes configurations d'un modèle de Machine Learning tout en observant la gestion des ressources de calcul.

6.2. Configuration de Ray Tune

L'implémentation nécessite de définir le modèle à entraîner, l'espace de recherche des hyperparamètres et la fonction chargée d'évaluer chaque configuration.

Ray Tune permet ensuite de lancer plusieurs essais et de gérer leur exécution en fonction des ressources disponibles, notamment les processeurs (CPU) et les processeurs graphiques (GPU), si l'environnement en dispose.

6.3. Utilisation d'ASHAScheduler

ASHAScheduler est un mécanisme de planification qui permet d'interrompre les essais dont les résultats intermédiaires sont peu prometteurs.

Son utilisation vise à limiter les calculs consacrés aux configurations moins performantes et à favoriser la poursuite des essais les plus intéressants.

L'expérimentation permettra d'observer le nombre d'essais interrompus, les performances obtenues et l'influence de ce mécanisme sur le temps d'exécution.

6.4. Utilisation de Population Based Training (PBT)

Population Based Training est une méthode qui fait évoluer les configurations d'hyperparamètres pendant l'entraînement. Elle compare plusieurs essais et peut modifier les configurations moins performantes en s'inspirant de celles qui donnent de meilleurs résultats.

Cette approche permet d'étudier une optimisation dynamique, dans laquelle les hyperparamètres ne restent pas nécessairement fixes pendant toute la durée de l'entraînement.

6.5. Exécution parallèle et gestion des ressources

Ray Tune permet d'exécuter plusieurs essais en parallèle, dans la limite des ressources disponibles. Le degré de parallélisation dépend notamment du nombre de CPU, des GPU disponibles et des ressources demandées par chaque essai.

L'expérimentation portera sur le nombre d'essais exécutés simultanément, le temps total d'exécution et l'utilisation des ressources.

6.6. Analyse des résultats

Les résultats obtenus avec Ray Tune seront analysés selon plusieurs critères :

- la meilleure performance du modèle ;
- le nombre d'essais exécutés et interrompus ;
- le temps total d'optimisation ;
- l'efficacité de la parallélisation ;
- l'utilisation des ressources disponibles.

Cette analyse permettra d'évaluer l'intérêt de Ray Tune pour la gestion d'expérimentations d'optimisation des hyperparamètres.

Les valeurs numériques, les graphiques et les observations seront ajoutés après l'exécution effective des programmes.

7. INTÉGRATION ET ANALYSE DES RÉSULTATS

7.1. Intégration d'Optuna dans Ray Tune

L'intégration d'Optuna dans Ray Tune permet de combiner les mécanismes de recherche d'hyperparamètres d'Optuna avec les capacités de gestion et d'exécution des essais de Ray Tune. Cette approche repose notamment sur **OptunaSearch**, qui permet d'utiliser une stratégie de recherche d'Optuna dans une expérimentation pilotée par Ray Tune.

L'objectif est de bénéficier d'une recherche guidée des configurations tout en facilitant l'exécution parallèle des essais.

[Insérer ici la capture du code d'intégration d'OptunaSearch dans Ray Tune.]

Figure 1 : Intégration d'OptunaSearch dans Ray Tune.

7.2. Visualisation de la convergence

La courbe de convergence permet de suivre l'évolution des performances au cours des essais d'optimisation. Elle aide à observer si les configurations testées permettent d'améliorer progressivement la métrique choisie.

Cette visualisation facilite l'identification des essais les plus performants et permet de comparer les stratégies de recherche utilisées.

[Insérer ici le graphique de convergence obtenu lors de l'expérimentation.]

Figure 2 : Évolution des performances au cours de l'optimisation.

7.3. Importance des hyperparamètres

L'analyse de l'importance des hyperparamètres vise à déterminer lesquels ont le plus d'influence sur la métrique étudiée. Cette information peut aider à mieux comprendre le comportement du modèle et à orienter les recherches futures.

L'importance dépend toutefois du modèle, des données et de l'espace de recherche utilisé. Elle doit donc être interprétée à partir des résultats effectivement obtenus.

[Insérer ici le graphique d'importance des hyperparamètres, s'il est disponible.]

Figure 3 : Importance des hyperparamètres dans l'optimisation.

7.4. Comparaison des résultats

La comparaison des expérimentations réalisées avec Optuna, Ray Tune et leur intégration permettra d'évaluer leurs performances respectives.

Les critères retenus sont notamment :

- la meilleure valeur de la métrique d'évaluation ;
- le temps total d'exécution ;
- le nombre d'essais réalisés ;
- le nombre d'essais interrompus, lorsque cette information est disponible ;
- l'utilisation des ressources de calcul.

Les résultats seront présentés dans un tableau comparatif complété après les expérimentations.

Critère	Optuna	Ray Tune	OptunaSearch avec Ray Tune
Meilleure performance	À compléter	À compléter	À compléter
Temps d'exécution	À compléter	À compléter	À compléter
Nombre d'essais	À compléter	À compléter	À compléter
Utilisation des ressources	À compléter	À compléter	À compléter

7.5. Discussion

L'analyse des résultats permettra de déterminer si l'exécution parallèle apporte un gain de temps dans l'environnement utilisé et si l'intégration des deux outils présente un avantage par rapport à leur utilisation séparée.

Il faudra tenir compte des caractéristiques de la machine, du nombre d'essais, des hyperparamètres testés et des conditions d'exécution. En effet, l'exécution distribuée ne garantit pas systématiquement une amélioration des performances du modèle ni une réduction du temps d'exécution, notamment lorsque les ressources disponibles sont limitées.

Les conclusions de cette analyse seront formulées à partir des mesures obtenues pendant le TP.

8. CONCLUSION

Ce travail pratique porte sur l'optimisation des hyperparamètres des modèles de Machine Learning à l'aide d'Optuna et de Ray Tune. Il vise à comprendre les mécanismes de recherche des meilleures configurations et à étudier l'intérêt de l'exécution parallèle et distribuée des expérimentations.

L'étude théorique a permis de présenter les principaux concepts liés à ces deux outils, notamment les samplers et les pruners d'Optuna, ainsi que les algorithmes de recherche et les schedulers de Ray Tune. L'implémentation pratique permettra d'observer leur fonctionnement à travers des essais d'optimisation et l'analyse des résultats obtenus.

L'intégration d'OptunaSearch dans Ray Tune constitue également un aspect important de ce travail, car elle permet d'associer une stratégie de recherche d'hyperparamètres aux mécanismes d'exécution et de gestion des ressources de Ray Tune.

La comparaison finale reposera sur les performances obtenues, le temps d'exécution, le nombre d'essais et l'utilisation des ressources disponibles. Elle permettra de mieux comprendre les avantages et les limites de chaque approche dans le contexte de l'expérimentation réalisée.

En définitive, ce TP met en évidence l'importance de l'automatisation de la recherche des hyperparamètres et l'intérêt potentiel de l'optimisation distribuée pour améliorer l'efficacité des expérimentations en Machine Learning. Les conclusions définitives devront être confirmées par les résultats réels des implémentations.

9. BIBLIOGRAPHIE

1. Akiba, T., Sano, S., Yanase, T., Ohta, T. et Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, p. 2623–2631.

2. Bergstra, J. et Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research, 13, p. 281–305.

3. Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J. E. et Stoica, I. (2018). Tune: A Research Platform for Distributed Model Selection and Training. arXiv:1807.05118.

4. Documentation officielle d'Optuna. Optuna Documentation. https://optuna.readthedocs.io/

5. Documentation officielle de Ray. Ray Tune: Hyperparameter Tuning. https://docs.ray.io/en/latest/tune/

6. Documentation officielle de Ray. OptunaSearch. https://docs.ray.io/en/latest/tune/api/doc/ray.tune.search.optuna.OptunaSearch.html